
Google Summer of Code 2024

[Tracker]



Project: Improve Tracker SPARQL developer experience by creating a "web IDE" for developing queries

GENERAL DETAILS

Name: Divyansh Jain

Time Zone: India (UTC+5.30)

University: Indian Institute of Technology, Roorkee (IIT Roorkee)

Expected Graduation Date: May 2026

Primary Email: divyanshjain.2206@gmail.com

Secondary Email: divyansh_j@cs.iitr.ac.in

Matrix ID: @demigod345:matrix.org

IRC Nick: Demigod

GitHub: [Demigod345](#)

GitLab: [Demigod](#)

Size of the Project: Medium (175 Hours)

Description

<https://gitlab.gnome.org/Teams/internship/project-ideas/-/issues/46>

<https://gitlab.gnome.org/GNOME/tracker/-/issues/421>

This proposal aims to create a web-based Integrated Development Environment (IDE) specifically for writing and executing SPARQL queries within Tracker.

This user-friendly interface will empower developers by allowing them to compose and edit multiline SPARQL queries directly in a code editor, eliminating the need for the traditional terminal approach. Once a query is written, it can be effortlessly executed via the HTTP SPARQL endpoint, and the results will be presented in a visually appealing format, enhancing readability and user experience.

By lowering the barrier to entry for newcomers, boosting developer productivity with visual editing, and fostering collaboration through easier query sharing, this web IDE promises to significantly improve the experience for those using libtracker-sparql to interact with RDF databases.

Mentors

1. @sthursfield (sthursfield@gnome.org)
2. @carlosg (carlosg@gnome.org)

What city and country will you reside in during the summer?

I will be residing in [Roorkee, Uttarakhand \(University\)](#) or [Ghaziabad, Uttar Pradesh \(Home\)](#) during the summer, depending on the summer break dates of my university.

What applications/libraries of GNOME will the proposed work modify or create?

This proposal seeks to introduce a user-friendly web interface for crafting and executing SPARQL queries within the Tracker environment. The interface will provide a dedicated editor for composing multiline SPARQL queries, allowing users to construct complex queries easily. Once constructed, queries can be seamlessly executed by sending them to the SPARQL endpoint via HTTP. Finally, the retrieved results will be presented in a clear and organised table format for efficient data visualisation.

To achieve this integration, the project proposes modifying the `libtracker-sparql` library to incorporate an option for the web IDE within the Tracker 3 endpoint. Essentially, this would create a web interface embedded directly within the Tracker application, offering users a convenient and centralised location for SPARQL query creation and execution.

What benefits does your proposed work have for GNOME and its community?

A web frontend for the Tracker endpoint empowers developers to query RDF databases by executing SPARQL queries via a user-friendly web interface and displaying the results more intuitively. This project offers significant benefits for GNOME and its community:

1. **Lower Barrier to Entry:** Many developers, especially those new to RDF or SPARQL, might be intimidated by the command line. A user-friendly web interface lowers the barrier to entry, making it easier for newcomers to learn and contribute to RDF-based GNOME projects.
2. **Increased Developer Productivity:** Composing and editing queries visually in a web interface is often faster and less error-prone than writing them in a terminal. This is further streamlined by features like syntax highlighting and storing past queries, significantly boosting developer productivity.
3. **Improved Collaboration and Transparency:** The web interface can make it easier to share and reproduce SPARQL queries. This improves the transparency and maintainability of open-source projects within GNOME that rely on RDF data.

In essence, this project empowers developers by making RDF querying more approachable, efficient, and collaborative. This can lead to a more active and engaged open-source community around GNOME's RDF-based projects.

How do you plan to achieve completion of your project?

ADDING OPTION FOR WEB IDE

The initial development phase focuses on enabling access to the web IDE from the terminal. All existing functionalities for the `tracker3` endpoint are written within the file `src/tracker/tracker-endpoint.c`. To integrate the web IDE option, we can simply add a new entry for it alongside the existing options within the static `GOptionEntry entries[]` array.

Also, in the function `sanity_check(void)`, we will check if `--web-ide` is implemented with `--http-port` only. Finally, to access web IDE on the same port as the HTTP endpoint, the command `tracker3 endpoint --http-port 1234 --web-ide ...` would have to be used. The possibility of serving the Web IDE on a different port is discussed in [this section](#).

```

1 static gboolean web_ide = FALSE;
2
3 static GOptionEntry entries[] = {
4     ...
5     { "web-ide", 'w', 0, G_OPTION_ARG_NONE, &web_ide,
6       N_("Web Interface for Tracker SPARQL"),
7       NULL
8     },
9     ...
10 };

```

```

1 static gboolean
2 sanity_check (void)
3 {
4     ...
5     if (web_ide && !http_port) {
6         /* TRANSLATORS: those are commandline arguments */
7         g_printerr ("%s\n", _("--web-ide can only be used with --http-port"));
8         return FALSE;
9     }
10    ...
11 }

```

Creating and validating the option for web IDE

SERVING THE STATIC WEB IDE

We'll create a handler function `server_callback_handle_get()` within `src/libtracker-sparql/remote/tracker-http-module.c` to manage GET requests received on the HTTP port.

This function will utilize the libsoup library to serve static `.html`, `.css`, and `.js` files, making decisions based on the request header. If the request header is `Accept: text/HTML`, the web IDE page will be served. Otherwise, default RDF data will be displayed, maintaining the current behaviour.

We'll explore the feasibility of implementing this feature using `g_signal_emit()` and `g_signal_connect()`, consulting with mentors for further guidance. The precise implementation details will be finalized through discussions with project mentors to ensure alignment with best practices and project standards.

```

1 static void
2 server_callback (SoupServer      *server,
3                  SoupMessage     *message,
4                  const char      *path,
5                  GHashTable      *query,
6                  SoupClientContext *client,
7                  gpointer         user_data)
8 {
9     ...
10    if (g_strcmp0 (method, SOUP_METHOD_GET) == 0) {
11        server_callback_handle_get(request);
12    }
13    ...
14 }
15
16 static void
17 server_callback_handle_get (TrackerHttpRequest *request)
18 {
19     /* If Header is 'Accept: text/HTML' then this function will serve
20      * static files index.html through libsoup. However if Header is
21      * 'Accept: rdf/data' then it would serve the default RDF data, which
22      * is how it is currently implemented.
23     */
24 }

```

CREATING WEB INTERFACE

The next major phase of the project would focus on creating the web interface. The web interface would consist of several features such as:

Query Editor

The query editor in the web IDE would enable the users to edit multiline SPARQL Queries and execute them via the HTTP Sparql Endpoint. The query editor would highlight the keywords in SPARQL making it more visually appealing. The execute button would send a POST request to the HTTP Endpoint which would accept only certain request headers such as `application/sparql-results+json`.

Displaying query results in tabular form

The response to the POST request will be in JSON format. We will then parse this JSON data and display the results in a visually appealing tabular format, enhancing data organization and readability.



**TRACKER
SPARQL IDE**

Documentation

Examples

```

1 # Give me all song resources, their title, and their album title
2 SELECT ?song ?songTitle ?albumTitle {
3   ?song a nmm:MusicPiece ;
4         nmm:musicAlbum ?album ;
5         nie:title ?songTitle .
6   ?album nie:title ?albumTitle
7 }
8
9
10
11
12
13
14

```

Set a parameter value

Execute

Results:

song	songTitle	albumTitle
example	example	example
example	example	example
example	example	example
example	example	example
example	example	example
example	example	example
example	example	example
example	example	example

Design Prototype for the Web Integrated Development Environment

```

1 function fillTable(data){
2   var table = document.getElementById("table");
3   table.innerHTML = "";
4   var HeaderRow = table.createTHead().insertRow(0);
5   const vars = data.head.vars;
6   for (var i = 0; i < vars.length; i++){
7     let th = document.createElement("th");
8     th.innerHTML = vars[i];
9     HeaderRow.appendChild(th);
10  }
11
12  const bindings = data.results.bindings;
13  for (var i = 0; i < bindings.length; i++){
14    var row = table.insertRow(i+1);
15    for (var j = 0; j < vars.length; j++){
16      let cell = row.insertCell(j);
17      cell.innerHTML = bindings[i][vars[j]].value;
18    }
19  }
20 }

```

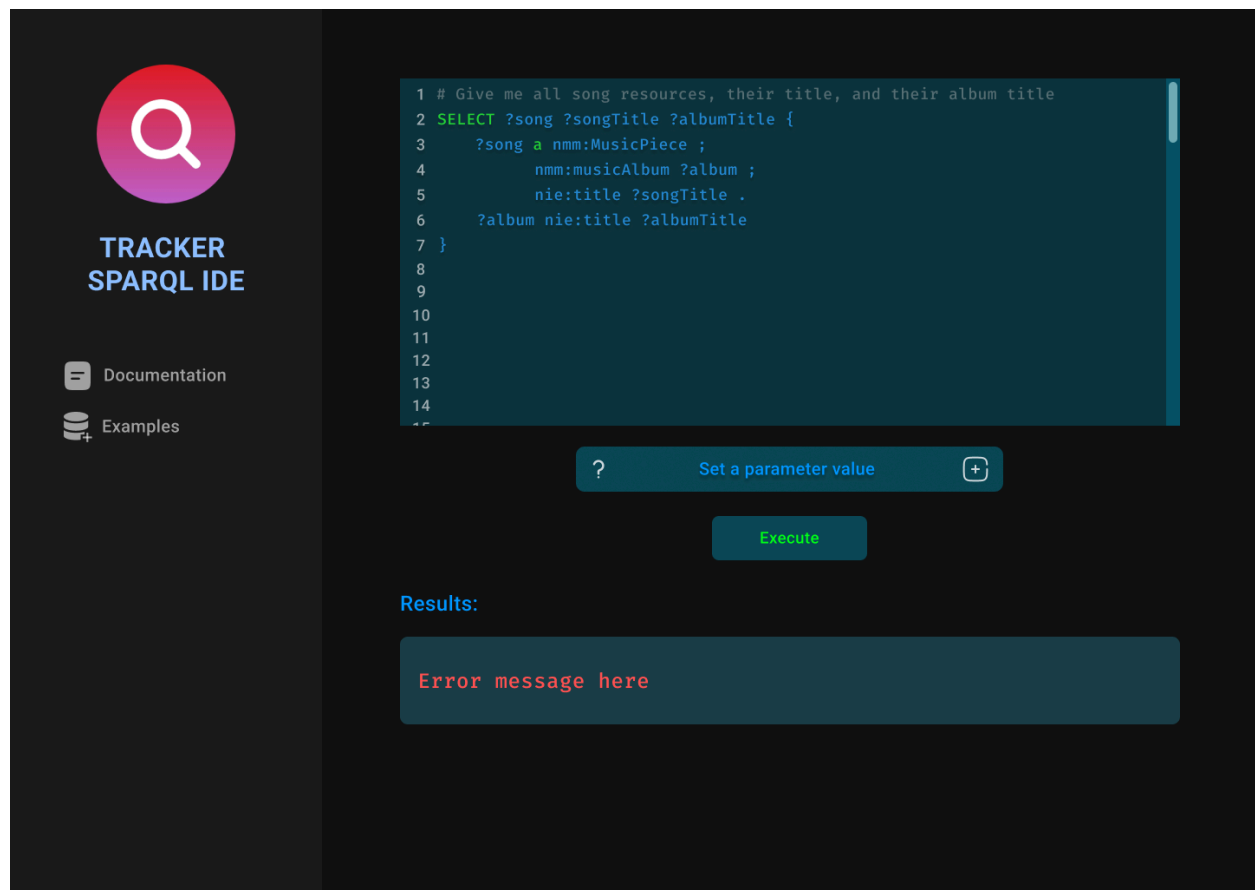
Implementation for generating a table from JSON data

Displaying all information for a given resource

In RDF triples, the subject and predicate typically use URLs to identify resources. To display all available information for a specific resource and enable navigation between resources, we can create hyperlinks from these URLs. Users can then click these links and be directed to dedicated pages displaying comprehensive details about the linked resource. The specific implementation details, such as the design and behaviour of these hyperlinks, can be further discussed with the mentors.

Error Highlighting

The initial plan focuses on displaying errors after clicking the "Execute" button. Upon sending a POST request to the HTTP endpoint, any bad query will trigger an error message response. This provides basic error highlighting.



Design Prototype for displaying error message

Documentation and Examples for New Users

To assist new users in testing and exploring the tool, we'll provide links to documentation and example queries. The link to the documentation will direct users to the [Tracker SPARQL tutorial](#),

offering a comprehensive guide to using the query language. The link to examples will lead users to a dedicated page (either hosted locally within the web IDE or on a separate webpage) showcasing a collection of example queries. These examples can be easily copied, pasted, and adapted to suit specific needs.

Setting parameter values

The queries might have a lot of variables. To streamline the process and significantly narrow down results, we'll introduce a user-friendly feature that allows setting values for specific parameters. This functionality will be implemented as a checklist, where users can define key-value pairs, making it easier to control query behaviour and filter data effectively.

```
1 # Give me all song resources, their title, and their album title
2 SELECT ?song ?songTitle ?albumTitle {
3   ?song a nmm:MusicPiece ;
4         nmm:musicAlbum ?album ;
5         nie:title ?songTitle .
6   ?album nie:title ?albumTitle
7 }
8
9
10
11
12
13
14
15
```

	Parameter Name	Parameter Value
☐	key	value
☐	key	value
☐	key	value

Execute

Results:

song	songTitle	albumTitle
example	example	example
example	example	example
example	example	example
example	example	example

FUTURE GOALS

Considering a Separate Port for the Web IDE

To minimize changes to `TrackerEndpointHttp`, we propose exploring a separate port for the web IDE. This could involve either a standalone soup server for dedicated web IDE request handling or potentially reusing an internal mechanism within `TrackerHttpServer` to run it on a different port. The `web_ide` variable would need to be changed to `gint` to accommodate the

port number, and a server launched on the designated port. However, specific implementation details require further investigation with mentor guidance.

Advanced Error Highlighting

Beyond basic post-execution highlighting, the project can optionally integrate a SPARQL linter in later stages. This linter would provide real-time feedback by identifying potential syntax errors during in-IDE query composition. Leveraging pre-execution analysis, the linter would proactively highlight errors directly in the UI. This advanced error handling would significantly enhance the development experience by promoting early detection of query syntax errors. Implementation details of the linter can be discussed with mentors for optimal integration.

Preserving Past Queries

This proposal explores the potential benefit of implementing a mechanism to save past queries (subject to available development time). This feature would be particularly valuable for users who work with long queries containing multiple parameters. Saved queries can be easily accessed and reused, improving workflow efficiency. The proposed implementation involves creating a dedicated RDF database to store these frequently used queries. Users can then "bookmark" or mark specific queries for later access.

All in all, this project will create a web interface for writing and running SPARQL queries in Tracker. This will make it much easier for developers to use Tracker, especially for those who are new to SPARQL. By making the process more user-friendly, this web interface will help developers get more out of Tracker. :)

Please provide a sequence of tasks and subtasks and how long (days) you estimate it will take you to complete each of them. Highlight important milestones/deliverables.

Pre GSOC Period	
April 3 - May 1	• I would read more about SPARQL queries and RDF databases to understand them better. I would also read more about Tracker and its functionalities. • I would continue working on other existing issues to improve my understanding of the project.
Community Bonding Period	
May 2 - May 26	• With the help of my mentor, I'll help to familiarize myself with the community and the codebase. • I'll discuss potential ideas to solve the issue.

	Work on different issues and bugs related to syncing to better transition into this project and hence, identify more relevant parts of the codebase.
Coding Period	
May 27 - June 5	Work on - Adding an option for web IDE. - Getting myself familiarized with the libsoup library.
June 6 - June 15	Work on - Serving static files on GET requests. - Serving different responses for GET requests depending upon Request Headers.
June 16 - June 25	Work on - Creating the Web Interface and implementing the following features: - Query Editor - Displaying query results in tabular form - Displaying all information for a given resource - Basic Error highlighting
June 26 - July 2	Work on - Remaining features of the Web Interface: - Documentation and Examples - Setting Parameter Values
July 3 - July 8	In case of no pending work, - Check for bugs and work on enhancing user experience - Prepare for mid-evaluation
Mid evaluation	
July 12 - July 24	- Continue work on the web interface - In case, the work is finished early, discuss potential implementation for the future goals
July 25 - August 5	Work on Implementing the features mentioned in future goals
August 6 - August 16	- Debugging and testing. - If completed early, discuss new features and gather feedback about the tool from the community.
August 17 - August 26	Work on Pending or last-minute issues and prepare for final evaluation.
Final evaluation	

What are your past experiences with the open-source world as a user and as a contributor?

As a user, I've had many experiences with open-source software. The first open-source project that I used even as a kid was VLC Player. I was amazed by the quality of the software, and how it was free for all to use and contribute to, and thus I decided to explore this world of open-source software further. Since last year, I have been using Ubuntu as my primary operating system, with GNOME as the desktop environment and have been constantly amazed by the quality of the software.

I was initially amazed by their free availability but soon realised the significance of contributing and giving back to the open-source community. Hence, I have been actively contributing to open-source software ever since I learnt programming in my 1st year University course. I have learnt a lot by working on these projects with the support of experienced developers and a helpful community.

I am an active member of [SDSLabs](#), a student-run technical group in my college and have contributed to various projects under the group, the details of which are given in the upcoming sections.

I have contributed to several projects under the GNOME Organization the details of which are listed below

- **Tracker:** To get familiarized with the codebase and explore various components of the Tracker Project, I made some contributions.
 - [Tracker-tag: fix man page](#) (merged)
This was the fix for [issue #274](#) which changed the man page of tracker3-tag to show the correct short form for –and-operator. Even though it was a small fix, it helped me to navigate and get familiar with the codebase.
 - [Build: add a feature option for fanotify](#) (draft)
This was related to [issue #302](#) which mentioned the need to always enable fanotify support on Linux. I learnt about build options while contributing to this issue.
- **Nautilus:** This was the first project under the GNOME organization that I contributed to. I learnt a lot since this was my first experience contributing to a project that was used by millions of people daily. Some of my contributions were:-
 - [Fix undo and redo action states in the window's main menu](#) (merged)

This was the fix for [issue #3263](#). This fixed the incorrect actions set for undo and redo in the UI window. This fix also set the state of Undo and Redo actions on constructing a new Window based on the current state. I learnt about the importance of commit messages and the art of navigating through large codebases, which might look intimidating at first.

- **[Window: Change issue-url for "Report An Issue" \(merged\)](#)**

This was related to [issue #3220](#), which mentioned the need to change the “Report an issue” URL so that all the users, regardless of whether they are logged in, can browse through the existing issues.

- **Crosswords:** I contributed to crosswords since I learned this project was relatively new and actively under development. I enjoyed working with a community of such active people.

- **[Disable multi char mode while viewing a completed puzzle \(merged\)](#)**

This was the fix for [issue #204](#). The fix required us to make the cell uneditable when the puzzle is in Browse mode.

- **Papers:** I have also contributed to GNOME Papers, the software for viewing documents. I was highly impressed by the efforts the community as a whole were making to improve the reading and rendering speed and were constantly trying to make it at par with proprietary software.

- **[Window: show shortcut for 100% zoom \(merged\)](#)**

This was the fix for [issue #92](#). We were required to show the shortcut for 100% Zoom in the menu Widget. This issue was a minor fix, but I learned about GObject and GTK through this.

Apart from the GNOME Organization, I have also contributed to various other open-source projects, which are mentioned in the [section below](#).

Why are you the right person to work on this project?

With more than a year of experience in developing full-stack applications with HTML, CSS, Javascript, Typescript, C/C++, Go and various other tech stacks, both as personal projects and contributing to large open-source software, I'm always eager to broaden my knowledge by diving into new technologies. I'm confident that my expertise can meet the high standards required in open-source organisations. I can work effectively within a team, provide and receive constructive feedback, and adapt to changing project requirements.

As an active GNOME contributor, I'm familiar with your work culture, codebase structure, and how to navigate the community for support. My current Web IDE prototype showcases my understanding of the project's goals and my ability to take the initiative.

Experience building production-level applications at SDSLabs has equipped me to handle technical challenges and adapt to evolving project requirements. My strong problem-solving skills, honed through academic challenges, ensure I can tackle complex tasks effectively.

I'm a highly motivated individual with a passion for learning new technologies. Given the opportunity to contribute to this GNOME project, I'm dedicated to making it exceptionally user-friendly and maximising its impact.

Please include links to your code contributions that have already been merged, or to Gitlab merge requests for the issues you fixed for the project of your proposal or any other GNOME projects. This demonstrates your willingness to learn and familiarity with development workflow.

1. https://gitlab.gnome.org/GNOME/tracker-miners/-/merge_requests/510 (merged)
2. https://gitlab.gnome.org/GNOME/tracker-miners/-/merge_requests/521 (open for reviews)
3. https://gitlab.gnome.org/GNOME/nautilus/-/merge_requests/1397 (merged)
4. https://gitlab.gnome.org/GNOME/nautilus/-/merge_requests/1403 (merged)
5. https://gitlab.gnome.org/jrb/crosswords/-/merge_requests/157 (merged)
6. https://gitlab.gnome.org/GNOME/Incubator/papers/-/merge_requests/127 (merged)

If available, please include links to any code you wrote for other open-source projects.

1. [Implement APU \[WIP\]](#) [SDSLabs/Gbemu]
2. [Timer Service](#) [SDSLabs/Athena]
3. [bugfix: set default user.time.left to duration of quiz](#) [SDSLabs/Athena]
4. [Minor bug fixes](#) [SDSLabs/Athena]
5. [fix: handle invalid user and quiz status](#) [SDSLabs/Helios]
6. [Format Client Dashboard](#) [SDSLabs/Helios]
7. [bugfix: fix hook useWindowFocus\(\)](#) [SDSLabs/Helios]

What other relevant projects have you worked on previously, and what knowledge have you gained from working on them?

I have worked on various projects, encompassing both frontend and backend fields. I have experience in Javascript (React, Next, Node, Express), Typescript, Golang, C++, Django, Ruby on Rails, Mongoose and MySQL. A few projects are listed below.

GBEMU:

[<https://github.com/sdslabs/gbemu>]

- Gbemu is a Game Boy emulator platform made using the C++ programming language, designed to evoke the nostalgic essence of the Game Boy device through precise emulation.
- Integrated audio support for various Game Boy games by utilising the SDL (Simple DirectMedia Layer) library and meticulously synchronised the Audio Processing Unit (APU) clock with the master CPU clock, ensuring an authentic gaming experience.

Skyview Capital:

[https://github.com/Demigod345/SkyView_Capital]

- A Stock Predictor System made using Deep Learning models which aims to provide accurate predictions of stock prices for a specified number of days into the future.
- Technology used: MongoDB, ExpressJs, ReactJs, NodeJs, Python, Flask, TensorFlow, PyTorch.
- Orchestrated the design and implementation of the system's backend architecture, meticulously optimising latency performance while upholding the highest standards of prediction accuracy.

Quizio:

[<https://github.com/sdslabs/Athena>]

[<https://github.com/sdslabs/Helios>]

- Quizio is an online platform for conducting exams and to conduct recruitments for students of IIT Roorkee. Frontend of the app is implemented in React Js, and the backend is in Node.js powered by the MongoDB database.
- I worked to implement the timer service that was authenticated from the backend, preventing any fraud and manipulation from the frontend.
- Got experience working on a large-scale CRUD app in use.

PRivate DEcentralised APPLication:

[https://github.com/EthOnline-2k23/PRivate_DEcentralised_APPLication]

-
- This project was developed for the EThOnline 2k23.
 - This app leverages a combination of proof of ownership via Zero Knowledge and the secure storage of transactions on L2 to facilitate private transactions within any blockchain network.
 - Developed the front-end and back-end for the project using NextJs.
 - Technologies used: Foundry, TypeScript, NextJs, Sismo ZK Authentication, TableLand, Scroll, Cometh, Push Protocol

5-staged Pipelined Processor:

- A custom simulator of a 64-bit Pipelined Processor made using C++.
- Implemented Data Memory, Instruction Memory and Register File along with the design of the ISA.
- It can simulate custom ISA built for optimal calculation of basic mathematical operations like add, sub, etc.

What other time commitments include school work, exams, research, another job, planned vacation, etc.? What are the dates for these commitments, and how many hours a week do these commitments take?

As a college student, I intend to utilise my summer breaks from May to July to work on the project. After completing my University exams in April, I will start working in May. I can dedicate 40 hours a week from May to July, while in August after the classes commence, I can dedicate about 25 hours a week.

There are no exams or planned vacations throughout the coding period. I generally work from 4 PM to 2 AM IST (6:30 AM to 4:30 PM Eastern Time), although I find it flexible to adjust my working hours if required.

Besides this project, I have no commitments/vacations planned for the summer. I shall keep my status posted to all the community members every week and maintain transparency in the project.

I am 100% dedicated to GNOME Tracker and have no plans to submit a proposal to any other organisation or project.